Progetto 2.1: **Georuggine: sistema di geolocalizzazione per una flotta di veicoli**

Il progetto consiste nella realizzazione di un'applicazione client/server sviluppata in Rust, finalizzata alla gestione della geolocalizzazione e della comunicazione con una flotta di veicoli. Il sistema deve permettere a più utenti di registrarsi, autenticarsi, inviare periodicamente la propria posizione al server e comunicare con esso tramite messaggi di testo.
Ogni utente accede al sistema attraverso una fase di registrazione, nella quale vengono definiti un account e una password. Una volta registrato, l'utente può essere monitorato dal server attraverso coordinate geografiche, gestite in modo emulato. La posizione di ciascun utente viene trasmessa al server ogni 30 secondi, così da consentire il tracciamento continuo degli spostamenti.
Il sistema deve inoltre gestire lo stato di ogni utente. In particolare, un utente può risultare sconnesso, fermo oppure in movimento. La transizione dallo stato "fermo" allo stato "in movimento" avviene quando viene rilevato un cambiamento delle coordinate. Al contrario, il passaggio dallo stato "in movimento" allo stato "fermo" si verifica quando, per almeno tre minuti, la posizione dell'utente non cambia.
Per simulare il movimento dei veicoli è possibile adottare diverse strategie. Una soluzione può consistere nella lettura di un file contenente una sequenza di coordinate e tempi. In alternativa, si può definire un punto di partenza e un punto di arrivo, specificando anche eventuali pause lungo il tragitto. Altre possibilità includono l'inserimento manuale delle coordinate tramite interfaccia oppure l'utilizzo di un generatore pseudo-casuale di posizioni.
Il server deve essere in grado di analizzare il movimento compiuto da uno specifico utente. Le informazioni richieste riguardano il tragitto percorso, la velocità media, la durata complessiva del movimento e la durata delle pause. Tali dati devono poter essere calcolati su intervalli temporali programmabili, come il giorno corrente, la settimana corrente o il mese corrente.
Un'altra funzionalità importante riguarda la comunicazione tra server e utenti. Il server deve poter inviare messaggi di testo sia in broadcast, cioè a tutti gli utenti connessi, sia in modo diretto a un singolo utente. Allo stesso tempo, ogni utente deve poter inviare messaggi testuali al server.
Dal punto di vista tecnico, l'applicazione deve essere eseguibile su almeno due piattaforme diverse tra Windows, Linux, macOS, Android, ChromeOS e iOS. È inoltre richiesto di prestare attenzione alle prestazioni del sistema, in particolare al consumo di tempo CPU e alla dimensione dell'applicativo. Il server deve generare un file di log che riporti, ogni due minuti, i dettagli relativi al tempo di CPU utilizzato. Infine, nel report descrittivo del progetto deve essere indicata anche la dimensione del file eseguibile prodotto.
In sintesi, il progetto richiede lo sviluppo di un sistema distribuito in Rust capace di gestire utenti, posizioni geografiche simulate, stati di movimento, analisi dei percorsi e comunicazione client/server. L'obiettivo è realizzare un'applicazione efficiente, multipiattaforma e ben strutturata, prestando attenzione sia agli aspetti funzionali sia a quelli prestazionali.

Progetto 2.2: YACCA RUST: preprocessing per l'irrobustimento del codice e Sviluppo di Ambiente di Fault Injection

Si chiede di realizzare un ambiente di fault injection per la valutazione della tolleranza ai guasti di un'applicazione che è stata irrobustita con l'inserimento di firme per la verifica della presenza di eventuali guasti che alterano il controllo del flusso (control flow error).

Il progetto mira a raggiungere 2 obiettivi:
1. Modificare il codice sorgente attraverso l'applicazione di un algoritmo di ridondanza del codice, descritto nel seguito
2. Realizzare un ambiente di Fault Injection costituito da:
   - un Programma ridondato come descritto più dettagliatamente nel documento allegati
   - una Lista di guasti caratterizzati dal nome della variabile, dal tempo di iniezione e dal bit modificato
   - un Iniettore che scandisce la lista di guasti e per ogni guasto lancia il programma in cui esso verrà iniettato
   - un Analizzatore che raccoglie i dati sulla base del risultato dell'elaborazione.

L'applicativo di preprocessing deve leggere il codice sorgente originale (scritto in RUST) e dovrà modificare il codice sorgente applicando la tecnica di ridondanza denominata YACCA (Yet Another Control-Flow Checking using Assertions), esempo di tecnica di SIHFT (Software-Implemented Hardware Fault Tolerance). In pratica, invece di aggiungere hardware ridondante, si aggiungono istruzioni software extra per rilevare errori durante l'esecuzione del programma.
Il suo scopo è quello di controllare che il programma segua il flusso di esecuzione previsto. Se il programma salta in un punto non valido, YACCA è in grado di rilevare l'errore.

Un programma normalmente segue un percorso preciso. Per esempio, se c'è un if, il programma può andare nel ramo then oppure nel ramo else, ma non dovrebbe saltare in punti casuali del codice.

Per spiegare il procedimento si consideri che un programma può essere suddiviso in basic block, ossia una sequenza di istruzioni con un solo punto di ingresso e un solo punto di uscita. Una volta entrato nel blocco, il programma dovrebbe eseguire le istruzioni in ordine fino alla fine del blocco.

Il programma può dunque essere rappresentato con un Control Flow Graph:

```
      B1
     /  \
  B2    B3
     \  /
      B4
```

In questo grafo, B1 puo andare a B2 o B3. Poi B2 e B3 possono andare a B4. Quindi, quando il programma entra in B4, i predecessori validi sono solo B2 e B3.

```
inizio -> B1 -> B2 -> B4
oppure
inizio -> B1 -> B3 -> B4
```

A causa di un errore hardware transitorio, come un bit flip nel program counter o nella memoria, il programma potrebbe eseguire un salto non previsto:

```
B1 -> B4   // salto diretto non consentito
B3 -> B2   // salto verso un blocco sbagliato
```

Questo tipo di problema si chiama Control Flow Error.

L'idea è aggiungere un controllo all'ingresso dei blocchi. Ogni basic block riceve un identificatore, per esempio:

```
B1 -> id 0
B2 -> id 1
B3 -> id 2
B4 -> id 3
```

Per ogni blocco si memorizzano i predecessori ammessi. Per B4, per esempio, i predecessori ammessi sono B2 e B3. Se il programma entra in B4 arrivando da B2 o B3, il flusso e corretto. Se entra in B4 arrivando da B1, viene rilevato un errore.

YACCA usa maschere di bit. Ogni blocco è rappresentato da un bit acceso in una parola binaria. Lo shift serve proprio ad accendere il bit corrispondente al blocco.

```
#define B1_ID 0
#define B2_ID 1
#define B3_ID 2
#define B4_ID 3

1 << 0  // 0001, rappresenta B1
1 << 1  // 0010, rappresenta B2
1 << 2  // 0100, rappresenta B3
1 << 3  // 1000, rappresenta B4
```

Quindi l'operazione 1 << ID significa: sposta il bit 1 nella posizione associata a quel basic block.

Per B4 i predecessori validi sono B2 e B3. La maschera dei predecessori validi si costruisce con OR bit a bit:

```
allowed_B4 = (1 << B2_ID) | (1 << B3_ID);
```

Sostituendo gli ID:

```
1 << 1 = 0010
1 << 2 = 0100
OR      = 0110
```

La maschera 0110 significa che B2 e B3 sono ammessi, mentre B1 e B4 non lo sono.

YACCA usa una variabile di controllo, spesso chiamata firma o signature, per ricordare qual e stato l ultimo blocco eseguito.

```
unsigned int signature;
```

Quando il programma termina un blocco, aggiorna la firma. Per esempio, se ha appena eseguito B2:

```
signature = 1 << B2_ID;   // signature = 0010
```

In questo modo, il blocco successivo puo controllare se arriva da un predecessore valido.

Quando il programma entra in B4, YACCA confronta la firma del blocco precedente con la maschera dei predecessori ammessi.

```
unsigned int allowed_B4 = (1 << B2_ID) | (1 << B3_ID);

if ((signature & allowed_B4) == 0) {
    error_handler();
}
```

La condizione usa AND bit a bit. Se il risultato e diverso da zero, significa che il blocco precedente e tra quelli ammessi. Se il risultato e zero, significa che il programma e arrivato da un blocco non valido.

Esempio corretto: arrivo da B2

```
signature  = 0010
allowed_B4 = 0110
AND        = 0010   // diverso da zero: corretto
```

Esempio corretto: arrivo da B3

```
signature  = 0100
allowed_B4 = 0110
AND        = 0100   // diverso da zero: corretto
```

Esempio errato: arrivo da B1

```
signature  = 0001
allowed_B4 = 0110
```

```
AND        = 0000   // zero: errore rilevato
```

Esempio completo, consideriamo questo programma:

```
if (x > 0) {
    y = y + 1;
} else {
    y = y - 1;
}

z = y * 2;
```

Lo dividiamo in quattro blocchi:

```
B1: controllo if
B2: ramo then
B3: ramo else
B4: codice dopo if
```

Una versione può essere:

```
#define B1_ID 0
#define B2_ID 1
#define B3_ID 2
#define B4_ID 3

unsigned int signature;

void error_handler(void) {
    while (1) {
        // errore rilevato
    }
}

void program(int x) {
    // Entrata in B1
    signature = 1 << B1_ID;

    if (x > 0) {
        unsigned int allowed_B2 = 1 << B1_ID;
        if ((signature & allowed_B2) == 0) {
            error_handler();
        }

        y = y + 1;
        signature = 1 << B2_ID;
    } else {
        unsigned int allowed_B3 = 1 << B1_ID;
        if ((signature & allowed_B3) == 0) {
            error_handler();
        }

        y = y - 1;
        signature = 1 << B3_ID;
```

```
        }

        unsigned int allowed_B4 = (1 << B2_ID) | (1 << B3_ID);
        if ((signature & allowed_B4) == 0) {
            error_handler();
        }

        z = y * 2;
        signature = 1 << B4_ID;
    }
```

Il codice originale serve a svolgere il calcolo principale. Il codice aggiunto da YACCA non serve direttamente al calcolo, ma serve a controllare che il programma stia eseguendo il percorso corretto.

```
    allowed = ...;
    if ((signature & allowed) == 0) {
        error_handler();
    }
    signature = ...;
```

Queste istruzioni sono ridondanza software: aumentano il codice e il tempo di esecuzione, ma permettono di rilevare errori nel flusso di controllo.

Ai gruppi che sceglieranno questo progetto saranno forniti i programmi di benchmark e ulteriori dettagli sull'algoritmo YACCA.